Infraestrutura de software
Guia de Relatorios

Atenção

- **Este relatório não é uma entrega separada: ele deve ser enviado junto com o .tar da atividade, no mesma atividade no Google Classroom.**
- **O relatório deve ter como nome as iniciais do seu e-mail em minúsculas (ex.: mla@cesar.school → diretório mla). Fora desse padrão, o relatório é descartado automaticamente.**
- **Cópia identificada, de outro(a) aluno(a) ou de qualquer outra fonte, descarta toda a submissão, código e relatório juntos.**

1. Objetivo (3–5 linhas)

O objetivo foi implementar a sincronização do metrô da METROREC, onde cada vagão e cada passageiro são threads separadas. Fiz a struct station e as funções station_init, station_load_car, station_wait_for_car e station_on_board usando só um mutex por estação e variáveis de condição, sem semáforo e sem espera ocupada. Pra controlar o embarque eu usei três contadores, passageiros esperando, assentos livres e passageiros embarcando.

2. Checklist de Requisitos

Um requisito por linha. Sem parágrafo, se precisar explicar algo, use a coluna "como testar".

Requisito do enunciado	Status	Como testar / evidência
Definir a struct station e as 4 funções	OK	sync_car.h e sync_car.c, o runner compila com elas
station_load_car só retorna com todo mundo sentado e vagão cheio ou sem ninguém esperando	OK	./sync_car nunca mostra "returned early"
station_wait_for_car só retorna com carro na estação e assento livre	OK	./sync_car e ./teste_extra, nenhum carro leva mais gente que cabe
station_on_board avisa o carro que o passageiro sentou	OK	O carro só sai depois do último on_board
Usar só as funções pthread pedidas	OK	grep -o "pthread_[a-z_]*" sync_car.c | sort -u
Não usar semáforo nem outra primrtiva	OK	grep -c sem_ sync_car.c dá 0
Só um lock por struct station	OK	sync_car.h tem um único pthread_mutex_t
Embarque simultaneo	OK	Contador boarding, o runner chama on_board só depois dos wait_for_car retornarem
Sem busy-wait	OK	Toda espera é pthread_cond_wait dentro de while
Compilar por Makefile gerando o executável sync_car	OK	make clean && make
struct station declarada dentro do sync_car.c	Parcial	Ficou no sync_car.h porque o runner do professor inclui esse header (seção 4)
Testar caso limite e caso inválido	OK	./teste_extra (carro com -3 assentos e carro sem ninguém esperando)
README e Makefile com limpeza na entrega	OK	README.md e make clean

3. Como Reproduzir

Comandos reais, copiáveis. Se não rodar com o que está aqui, não é avaliado.

Compilar:

```
cd ~/ebs2
make clean && make
```

Executar (exemplo):

```
./sync_car
for i in $(seq 1 100); do ./sync_car > /dev/null 2>&1 && echo ok; done | wc -l
gcc -Wall -pthread -o teste_extra teste_extra.c sync_car.c
./teste_extra
```

4. Arquitetura (resumida)

Uma linha por arquivo, só o que é específico do seu projeto.

Arquivos e responsabilidades:

1. sync_car.h -> Define a struct station (o mutex, as duas variáveis de condição car_arrived e all_boarded e os contadores waiting, free_seats e boarding) e os protótipos das funções
2. sync_car.c -> Implementa station_init, station_load_car, station_wait_for_car e station_on_board
3. sync_car-runner.c -> Já tem o main. Não mexi nele
4. teste_extra.c -> Teste que eu fiz pra casos que o runner não cobre (assento negativo e carro sem ninguém esperando)
5. Makefile -> Compila o runner junto com o sync_car.c e gera o sync_car, e o make clean apaga o executável

Decisões técnicas:

Decisão -> Motivo -> Consequência

1. Colocar a struct no sync_car.h -> O runner faz #include "sync_car.h" e cria a struct station direto no main, então precisa dela completa -> Fugiu um pouco do enunciado, que pedia a struct no sync_car.c, mas sem isso o teste oficial nem compila
2. Contador boarding separado de free_seats -> Vários passageiros precisam poder sair do wait_for_car antes de qualquer on_board -> O carro só vai embora quando boarding chega em 0, ou seja, quando todo mundo ja ta sebtado
3. Reservar o assento dentro do wait_for_car com o lock pego -> Evitar que dois passageiros peguem o mesmo assento -> Nenhum carro leva mais gente do que os assentos que tem
4. broadcast pros passageiros e signal pro carro -> Quando o carro chega vários passageiros podem embarcar, mas só tem um carro esperando -> Todo mundo que pode embarcar acorda e o carro só acorda quando da pra sair
5. pthread_cond_wait sempre dentro de while -> A thread pode acordar sem a condição estar valendo -> A condição é checada de novo e não tem espera ocupada
6. Zerar free_seats quando o carro sai -> Passageiro que chega depois não pode entrar num carro que já foi embora -> Isso também resolve o caso de assento negativo, o carro só sai vazio

5. Estratégias e Diário de Desenvolvimento

Isto é o núcleo do relatório.

5.1 Estratégias da semana

Uma linha por abordagem diferente que você tentou. Se você não mudou de abordagem, preencha só S1.

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Tudo no sync_car.c	A primeira ideia foi colocar a struct, as 4 funções e um main meu de testes tudo no mesmo arquivo, que era o que o enunciado parecia pedir	Mudei quando vi o sync_car-runner.c, que inclui um sync_car.h e já tem o próprio main, então do jeito da S1 não ia compilar
S2	Header + runner	Passei a struct e os protótipos pro sync_car.h, tirei o meu main do sync_car.c e comecei a testar com o runner	
S3			

5.2 Diário de Tentativas

Uma linha por tentativa relevante, marcando a qual estratégia (S1/S2/S3) ela pertence, obviamente precisa ser em ordem cronológica. A coluna "Quando" precisa bater com a ordem cronológica do evidencias.log, é isso que comprova que o fluxo é real. "Hipótese/Causa" só é preenchida quando o resultado foi falha ou parcial.

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	s1	Fiz a primeira versão com a struct, as 4 funções e um main com testes meus, tudo no sync_car.c	certo		03/11	Commit
2	s1	Fui usar o sync_car-runner.c	falhou	O runner inclui um sync_car.h que eu não tinha e já tem main, então ia dar dois main	03/11	Commit
3	s2	Separei a struct e os protótipos no sync_car.h e tirei o meu main	certo		03/11	Commit
4	s2	Tentei rodar o make no PowerShell do Windows	falhou	O Windows não tem make e o runner usa alarm/SIGALRM, que só existe em Linux/Unix	03/11	Print
5	s2	Mudei pro Ubuntu 24.04 no WSL e rodei apt update e build-essential	certo		03/11	Print
6	s2	Criei o sync_car.c	falhou	O texto embaralhou no meio da colagem e o terminal ficou travado esperando o EOF	03/11	Commit
7	s2	Preparei o arquivo em 4 partes e conferi com wc -l e md5sum	certo		03/11	Saída do md5sum
8	s2	Ajeitei o runner do Windows, criei o Makefile e compilei com make	certo		03/11	Print
9	s2	Rodei o ./sync_car	certo		03/11	Print
10	s2	Rodei o teste 100 vezes seguidas	certo		04/11	Print
11	s2	Corrigi o README	certo		04/11	Commit
12	s2	Subi os arquivos no GitHub	certo	O runner subiu como sync_car-runner (1).c por causa do download, aí renomeei (tinha falhado, mas já ajeitei, por isso botei certo)	04/11	Histórico do GitHub

Regra prática: se algo deu muito errado (bug que te travou por horas), a linha da tabela resume em 1 frase e aponta para o trecho do evidencias.log com o detalhe.

6. Evidências

A regra continua: toda afirmação relevante precisa de prova. O que muda é como capturar essa prova, para não depender de print manual toda hora.

6.1 Log automático (faz o trabalho pesado por você)

No início da sessão de trabalho, rode:

```
script -a evidencias.log
```

Isso grava automaticamente todos os comandos e saídas do terminal num arquivo, sem você precisar tirar print de cada linha. Comece a sessão com date, whoami e pwd, assim o log já tem timestamp, usuário e diretório sem esforço extra. **O evidencias.log deve ser enviado dentro do .tar**

6.2 Prints

Testes que funcionaram:

```
dubox@DuPc:~/ebs2$ ./sync_car
car entering station with 36 free seats
car departed station with 36 new passenger(s) (expected 36)
car entering station with 45 free seats
car departed station with 45 new passenger(s) (expected 45)
car entering station with 31 free seats
car departed station with 19 new passenger(s) (expected 19)
Looks good!
```

```
dubox@DuPc:~/ebs2$ md5sum teste_extra.c
96572e9572381e99867d239b5b1067e5  teste_extra.c
dubox@DuPc:~/ebs2$ script -a evidencias.log
Script started, output log file is 'evidencias.log'.
dubox@DuPc:~/ebs2$ date && whoami && pwd
Wed Nov  4 18:54:28 -03 2026
dubox
/home/dubox/ebs2
dubox@DuPc:~/ebs2$ make clean && make && ./sync_car
rm -f sync_car
gcc -Wall -pthread -o sync_car sync_car-runner.c sync_car.c
car entering station with 27 free seats
car departed station with 27 new passenger(s) (expected 27)
car entering station with 11 free seats
car departed station with 11 new passenger(s) (expected 11)
car entering station with 10 free seats
car departed station with 10 new passenger(s) (expected 10)
car entering station with 37 free seats
car departed station with 37 new passenger(s) (expected 37)
car entering station with 3 free seats
car departed station with 3 new passenger(s) (expected 3)
car entering station with 43 free seats
car departed station with 12 new passenger(s) (expected 12)
Looks good!
```

```
dubox@DuPc:~/ebs2$ for i in $(seq 1 100); do ./sync_car > /dev/null 2>&1 && echo ok; done | wc -l
100
```

```
dubox@DuPc:~/ebs2$ gcc -Wall -pthread -o teste_extra teste_extra.c sync_car.c && ./teste_extra
Caso invalido (carro com -3 assentos): embarcaram 0 (esperado 0)
Caso limite (carro com 2 assentos, 5 esperando): embarcaram 2 (esperado 2)
Caso limite (carro com 3 assentos, 3 esperando): embarcaram 5 (esperado 5)
Caso limite (carro com 4 assentos, ninguem esperando): embarcaram 5 (esperado 5)
Testes extras OK
```

Erros:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\User>    make
make : O termo 'make' não é reconhecido como nome de cmdlet, função, arquivo de script ou programa operável. Verifique
a grafia do nome ou, se um caminho tiver sido incluído, veja se o caminho está correto e tente novamente.
No linha:1 caractere:4
+    make
+    ~~~~
    + CategoryInfo          : ObjectNotFound: (make:String) [], CommandNotFoundException
    + FullyQualifiedErrorId : CommandNotFoundException
```

7. Uso de IA

Usei o Claude como apoio durante o trabalho. Usei pra interpretar o enunciado e montar um plano de arquivos. Quando eu mandei o runner de Marcelo, que eu tinha esquecido de mandar no começo, usei o Claude pra entender o que mudava na estrutura do projeto. Também usei pra me orientar no WSL e pra revisar o relatório.

Prompts principais seriam me ajudar a montar um plano de arquivos com base no enunciado, me ajudar com o sync_car.c, me ajudar a entender a questão de novo depois que eu mandei o runner, me orientar nos comandos do WSL e me ajudar a embelezar o relatório.

E quanto a validação manual, eu mesmo montei os arquivos no meu Ubuntu pelo WSL e conferi com md5sum se cada um ficou certo. Compilei e rodei tudo, vi o Looks good! aparecer e rodei o teste 100 vezes, passando todas.

8. Reflexão Final (5–8 linhas)

O que você melhoraria com mais tempo, principais decisões técnicas e erros que evitaria na próxima. Tudo junto, sem subseções separadas.

Reflexão:

A principal decisão técnica foi separar os assentos livres (free_seats) dos passageiros que ainda tão embarcando (boarding), porque é isso que deixa vários passageiros embarcarem ao mesmo tempo e não deixa o carro sair com alguém ainda indo sentar. Na próxima vez eu tenho que prestar mais atenção no que Marcelo envia, fui muito afobado e acabei tendo mais problemas porque o runner mudou a estrutura do projeto que eu já tinha começado. Também busco depender cada vez menos de IA pra escrever o código, por mais que eu esteja cada vez mais usando menos.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	SIM
Rodei os testes e evidencias.log foi gerado	SIM
Tenho prints obrigatórios	SIM
Testei pelo menos um caso limite e um caso inválido	SIM
Preenchi a seção de uso de IA	SIM
Revisei o relatório e removi frases genéricas / vazias	SIM

10. Se eu tivesse mais 2 horas

Aplique as instruções abaixo de forma objetiva e verificável, evitando texto genérico.

Escreva 5-10 linhas sobre o que você melhoraria (robustez, testes, warnings, performance, organização).

Eu colocaria o teste_extra dentro do Makefile, com um make test, pra rodar todos os testes com um comando só. Também verificaria o retorno das funções pthread em vez de assumir que sempre dão certo, e criaria uma station_destroy pra liberar o mutex e as variáveis de condição. Trataria o caso de assento negativo de forma explícita, em vez de depender do carro sair vazio. Além disso eu faria testes com muito mais passageiros pra ver o desempenho do broadcast.

Regras de Integridade (não mudam)

- Ocorrência relevante sem evidência (log ou print) = não conta para a nota.
- Contradição entre prints, log e relato = relatório perde credibilidade.
- Indício de evidência fabricada = entrega invalidada.